

数据流和运算符通过流处理引擎进行管理和协调，以实现高效的数据处理。

流处理架构的典型应用场景包括实时数据分析、实时监控和实时推荐等。流处理架构的优点在于其能够实时处理数据，快速响应用户请求，同时也能够适应数据变化的快速性。但是，流处理架构也有其局限性。例如，无法对历史数据进行处理，因为数据是无限制的，处理器可能会漏掉某些数据，而且流式处理还需要考虑并发性和一致性问题。

3. 混合架构

混合系统架构是指同时使用批处理和流处理的架构。混合系统结合了批处理和流处理架构的优点，可以在需要时处理历史数据，同时也可以处理实时数据。混合系统架构可以使用像 Apache Spark 这样的引擎来实现，该引擎同时支持批处理和流处理。

在混合系统架构中，数据可以根据需要进行批处理或流处理。批处理通常用于处理历史数据或需要对数据进行全面分析的情况，而流处理通常用于实时处理和监控。在混合系统中，数据可以根据需要在不同的模式之间进行转换。

混合系统架构的优点在于能够兼顾批处理和流处理架构的优点，可以处理历史数据和实时数据，并且能够快速响应用户请求。但是，混合系统架构需要在架构和数据处理上进行权衡，并且需要付出更多的管理和维护成本。

19.2.3 大数据处理系统架构模式

1. Lambda 架构

Lambda 架构是一种将批处理和流处理结合起来的大数据处理系统架构模式，它旨在解决传统批处理架构的延迟问题和流处理架构的准确性问题。Lambda 架构是大数据平台里最成熟、最稳定的架构，它的核心思想是：将批处理作业和实时流处理作业分离，各自独立运行，资源互相隔离。图 19-3 展示了 Lambda 的基本架构。

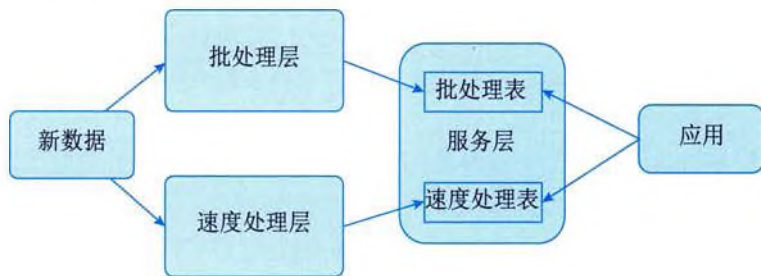


图 19-3 Lambda 架构图

Lambda 架构将数据流分为三个层次：批处理层（batch layer）、加速层（speed layer）和服务层（serving layer），这些层次各自具有不同的特性和用途。

(1) 批处理层。批处理层主要负责所有的批处理操作，支撑该层的技术以 Hive、Spark-SQL 或 MapReduce 这类批处理技术为主。批处理层既可以存储整个数据集，又能够计算出批处理的视图。由于此处的存储数据集不可被改变，因此只能被追加。也就是说，新的数据会不断地被传入，而原有旧的数据则会始终保持不变。同时，批处理层会通过对整个数据集的查询，

或功能性计算，得出各种视图。查询这些视图时，虽然可以在整个数据集中低延迟地找到答案，但是其缺点是系统需要花费大量的时间进行计算。

(2) 加速层。加速层使用流式计算技术实时处理当前数据，支撑该层的技术以 Storm、Spark Streaming 或 Flink 等这类流处理技术为主。加速层区别于批处理层的地方在于，尽管批处理层从开始就保留了所有数据，但是加速层仅关心从最后一批视图完成以来到达的数据。也就是说，加速层通过处理那些批处理视图尚未计入的最新数据查询，来弥补计算视图时的高延迟。但是，加速层的局限在于只能处理当前新生成的数据，无法对全部历史数据进行操作，因为流式计算只能针对当前产生的“热数据”进行处理。

(3) 服务层。以批处理层处理的结果数据为基础，对外提供低延时的数据查询和 ad-hoc 查询（即席查询）服务，服务层可以认为是对批处理层数据访问能力上的延伸或增强。因为批处理本身是比较慢的，无法支撑实时的查询请求，从服务层的角度看，批处理层的工作本质是一种“预计算”，即预先对大体量数据集进行处理，得到相对较小的结果集，然后由服务层接手，提供实时的数据查询服务。服务层既可以使用包括关系型数据库在内的传统技术，也可以使用 Kylin、Presto、Impala 或 Druid 等大数据 OLAP 产品。

Lambda 架构的优点在于能够同时支持批量处理和实时流处理，具有高吞吐量和处理多样化数据源的能力。同时，Lambda 架构提供了实时数据处理的能力，使用户可以快速获得数据分析结果。但 Lambda 架构也存在一些缺点，如需要维护多个层次的数据存储和复杂的数据整合，增加了系统复杂性和维护成本。此外，由于数据需要同时存储在批处理层和速度层，会造成数据冗余和增加存储成本。同时，Lambda 架构的实时性有限，无法应对对实时性要求极高的处理场景。Lambda 架构适用于需要处理大量数据，需要同时支持批量处理和实时处理的场景，但也需要在系统复杂性和实时性等方面做出权衡。

2. Kappa 架构

随着 Flink 等流式处理引擎的不断完善，流处理技术日益成熟。针对 Lambda 架构存在的不足，LinkedIn 的 Jay Kreps 结合实际经验和亲身体会提出了 Kappa 架构。Kappa 架构的核心思想是通过改进流计算系统来解决数据全量处理的问题，使得实时计算和批处理过程使用同一套代码。此外，Kappa 架构认为只有在必要时才会对历史数据进行重复计算，而如果需要重复计算时，Kappa 架构下可以启动很多个实例进行重复计算，方式是通过上游重放完成（从数据源拉取数据重新计算）。Kappa 架构就是基于流来处理所有数据的，天然的流计算分布式特征，注定了它的扩展性更好，通过加大流计算的并发性，加大流式数据的“时间窗口”，来统一批处理与流式处理两种计算模式。图 19-4 展示了 Kappa 的基本架构。

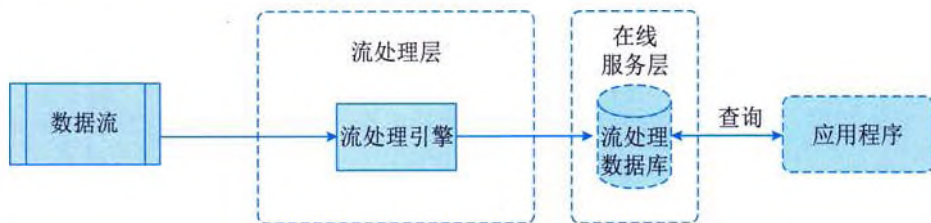


图 19-4 Kappa 架构图

Kappa 架构将数据流分为两个层次：流处理层和在线服务层。这些层次各自具有不同的特性和用途。

(1) 流处理层。该层级负责对实时流数据进行处理和计算，并将计算结果存储到实时数据库或分布式文件系统中。在 Kappa 架构中，流处理引擎采用无状态的流处理算法，即不保存中间结果，从而简化了数据处理逻辑和减少了计算成本。

(2) 在线服务层。该层级负责存储流处理引擎处理后的实时数据和计算结果，通常采用实时数据库或分布式文件系统。在 Kappa 架构中，在线服务层既可以存储实时数据，也可以存储历史数据。

Kappa 架构的优点是简化了系统架构和维护成本，提高了实时性和可伸缩性，同时能够对实时流数据进行处理和计算，并将计算结果存储到实时数据库或分布式文件系统中，适用于对实时性要求较高的场景。Kappa 架构的缺点是无法支持批处理和离线分析，在一些场景下可能需要离线处理大量历史数据，同时由于 Kappa 架构只有一个流处理层，数据存储成本可能会较高，需要仔细考虑数据存储的策略。

3. IOTA 架构

IOTA 架构是一种新兴的大数据处理系统架构，它强调数据流的连续性和一致性，可以满足对数据一致性要求更高的场景。IOTA 的整体思路是设定标准数据模型，通过边缘计算技术把所有的计算过程分散在数据产生、计算和查询过程当中，以统一的数据模型贯穿始终，从而提高整体的计算效率，同时满足计算的需要，可以使用各种即时查询（Ad-hoc Query）来查询底层数据。图 19-5 展示了 IOTA 的基本架构。

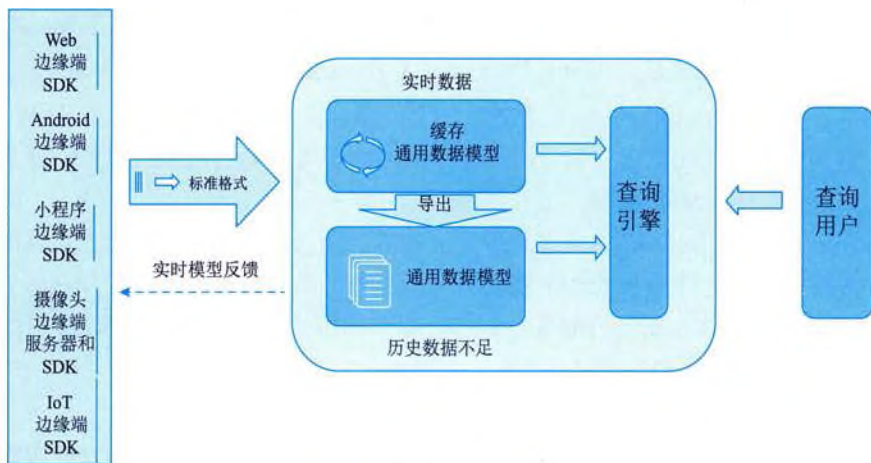


图 19-5 IOTA 架构图

IOTA 整体技术结构分为以下几部分：

(1) 通用数据模型（Common Data Model）。贯穿整体业务始终的数据模型，这个模型是整个业务的核心，要使 SDK、cache、历史数据、查询引擎保持一致。对于用户数据分析来讲，可以定义为“主—谓—宾”或者“对象—事件”这样的抽象模型来满足各种各样的查询。以大家熟悉的 App 用户模型为例，用“主—谓—宾”模型描述就是“X 用户—事件 1—A 页面

(2018/4/11 20:00)”。当然，根据业务需求的不同，也可以使用“产品—事件”“地点—时间”模型等。模型本身也可以根据协议（如 protobuf）来实现 SDK 端定义，中央存储的方式。此处的核心是，从 SDK 到存储再到处理是统一的一个通用数据模型。

(2) 边缘开发环境和边缘服务器（Edge SDKs & Edge Servers）。这是数据的采集端，不仅是过去简单的 SDK，在复杂的计算情况下，会赋予 SDK 更复杂的计算，在设备端就转化为形成统一的数据模型来进行传送。例如，对于智能 Wi-Fi 采集的数据，从 AC 端就变为“X 用户的 MAC 地址—出现—A 楼层（2018/4/11 18:00）”这种“主—谓—宾”结构，对于摄像头会通过 Edge AI Server，转化成为“X 的 Face 特征—进入—A 火车站（2018/4/11 20:00）”。也可以是上面提到的简单的 App 或者页面级别的“X 用户—事件 1—A 页面（2018/4/11 20:00）”，对于 App 和 H5 页面来讲，没有计算工作量，只要求埋点格式即可。

(3) 实时数据缓存区（Real Time Data）。实时数据缓存区是为了达到实时计算的目的。海量数据接收不可能实时存入历史数据库，那样会出现建立索引延迟、历史数据碎片文件等问题。因此，需要一个实时数据缓存区来存储最近几分钟或者几秒钟的数据。该部分可以使用 Kudu 或者 HBase 等组件来实现。这部分数据会通过 Dumper 来合并到历史数据当中。此处的数据模型和 SDK 端数据模型是保持一致的，都是通用数据模型。

(4) 历史数据沉浸区（Historical Data）。这部分是保存了大量的历史数据，为了实现 Ad-hoc 查询，将自动建立相关索引提高整体历史数据查询效率，从而实现秒级复杂查询百亿条数据的反馈。例如，可以使用 HDFS 存储历史数据，此处的数据模型依然是与 SDK 端数据模型保持一致的通用数据模型。

(5) 数据导入组件（Dumper）。数据导入组件的主要工作就是把最近几秒钟或者几分钟的实时数据，根据汇聚规则、建立索引，存储到历史存储结构中，可以使用 MapReduce、C、Scala 来编写，把相关的数据从 Realtime Data 区写入 Historical Data 区。

(6) 查询引擎（Query Engine）。提供统一的对外查询接口和协议（如 SQL JDBC），把实时数据和历史数据合并到一起查询，从而实现对于数据的即时查询。例如，常见的计算引擎可以使用 Presto、Impala、Clickhouse 等。

(7) Realtime model feedback：通过边缘计算技术，在边缘端有更多的交互可以做，可以通过在实时数据缓存区设定规则来对边缘 SDK 端进行控制。例如，数据上传的频次降低、语音控制的迅速反馈，某些条件和规则的触发，等等。简单的事件处理，将通过本地的 IOT 端完成。例如，嫌疑犯识别现在已经有摄像头本身带有此功能。

IOTA 大数据架构主要有如下几个特点：

(1) 去 ETL 化。ETL 和相关开发一直是大数据处理的痛点，IOTA 架构通过通用数据模型的设计，专注在某个具体领域的计算，从而可以从 SDK 端开始计算，中央端只做采集、建立索引和查询，提高整体数据分析的效率。

(2) 即时查询。鉴于整体的计算流程机制，在手机端、智能 IOT 事件发生之时，就可以直接传送到云端进入实时数据缓存区，可以由前端的查询引擎来查询。此时，用户可以使用各种各样的查询，直接查到前几秒发生的事件，而不用再等待 ETL 或者 Streaming 的数据研发和处理。